Переменные, Инициализация и Присваивание

Программируя на языке C++, мы создаем, обрабатываем и уничтожаем объекты. **Объект** — это часть памяти, которая может хранить значение. В качестве аналогии мы можем использовать почтовый ящик, куда мы помещаем информацию и откуда её извлекаем. Все компьютеры имеют **оперативную память**, которую используют программы. При создании объекта, часть оперативной памяти выделяется для этого объекта. Большинство объектов, с которыми мы будем работать в языке C++, являются переменными.

Оглавление:

1. Переменные
2. l-values и r-values
3. Инициализация vs. Присваивание
4. Неинициализированные переменные
5. Тест
6. Ответы

Переменные

Стейтмент `a = 8;` выглядит довольно простым: мы присваиваем значение `8` переменной `a`. Но что такое `a`? `a` — это переменная, объект с именем.

На этом уроке мы рассмотрим только целочисленные переменные. **Целое число** — это число, которое можно записать без дроби, например: -11, -2, 0, 5 или 34.

Для создания переменной используется стейтмент объявления (разницу между объявлением и определением переменной мы рассмотрим несколько позже). Вот пример объявления целочисленной переменной `a` (которая может содержать только целые числа):

```
int a;
```

При выполнении этой инструкции центральным процессором часть оперативной памяти выделяется под этот объект. Например, предположим, что переменной `a` присваивается ячейка памяти под номером 150. Когда программа видит переменную `a` в выражении или в стейтменте, то она понимает, что для того, чтобы получить значение этой переменной, нужно заглянуть в ячейку памяти под номером 150.

Одной из наиболее распространенных операций с переменными является операция присваивания, например:

```
a = 8;
```

Когда процессор выполняет эту инструкцию, он понимает её как «поместить значение 8 в ячейку памяти под номером 150».

Затем мы сможем вывести это значение на экран с помощью std::cout:

```
std::cout << a; // выводим значение переменной a (ячейка памяти под номером 150) на экран
```

l-values и r-values

В языке C++ все переменные являются l-values. **l-value** (в переводе *«л-значение»*, произносится как *«ел-валью»*) — это значение, которое имеет свой собственный адрес в памяти. Поскольку все переменные имеют адреса, то они все являются l-values (например, переменные `a`, `b`, `c` — все они являются l-values). І от слова «left», так как только значения l-values могут находиться в левой стороне в операциях присваивания (в противном случае, мы получим ошибку). Например, стейтмент `9 = 10;` вызовет ошибку компилятора, так как 9 не является l-value. Число 9 не имеет своего адреса в памяти и, таким образом, мы ничего не можем ему присвоить (9 = 9 и ничего здесь не изменить).

Противоположностью l-value является r-value (в переводе *«р-значение»*, произносится как *«ер-валью»*). **r-value** — это значение, которое не имеет постоянного адреса в памяти. Примерами могут быть единичные числа (например, `7`, которое имеет значение `7`) или выражения (например, `3 + x`, которое имеет значение `x` плюс `3`).

Вот несколько примеров операций присваивания с использованием r-values:

```
int a;      // объявляем целочисленную переменную a
a = 5;      // 5 имеет значение 5, которое затем присваивается переменной a
a = 4 + 6;  // 4 + 6 имеет значение 10, которое затем присваивается переменной a

int b;      // объявляем целочисленную переменную b
b = a;      // a имеет значение 10 (исходя из предыдущих операций), которое затем присваивается переменной b
b = b;      // b имеет значение 10, которое затем присваивается переменной b (ничего не происходит)
b = b + 2;  // b + 2 имеет значение 12, которое затем присваивается переменной b
```

Давайте детально рассмотрим последнюю операцию присваивания:

```
b = b + 2;
```

Здесь переменная `b` используется в двух различных контекстах. Слева `b` используется как l-value (переменная с адресом в памяти), а справа `b` используется как r-value и имеет отдельное значение (в данном случае, `12`). При выполнении этого стейтмента, компилятор видит следующее:

```
b = 10 + 2;
```

И здесь уже понятно, какое значение присваивается переменной `b`.

Сильно беспокоиться о l-values или r-values сейчас не нужно, так как мы еще вернемся к этой теме на следующих уроках. Всё, что вам нужно сейчас запомнить — это то, что в левой стороне операции присваивания всегда должно находиться l-value (которое имеет свой собственный адрес в памяти), а в правой стороне операции присваивания — r-value (которое имеет какое-то значение).

Инициализация vs. Присваивание

В языке C++ есть две похожие концепции, которые новички часто путают: присваивание и инициализация.

После объявления переменной, ей можно **присвоить** значение с помощью оператора присваивания (знак равенства `=`):

```
int a; // это объявление переменной
a = 8; // а это присваивание переменной a значения 8
```

В языке C++ вы можете объявить переменную и присвоить ей значение одновременно. Это называется **инициализацией** (или ***«определением»***).

```
int a = 8; // инициализируем переменную a значением 8
```

Переменная может быть инициализирована только после операции объявления.

Хотя эти два понятия близки по своей сути и часто могут использоваться для достижения одних и тех же целей, все же в некоторых случаях следует использовать инициализацию, вместо присваивания, а в некоторых — присваивание вместо инициализации.

Правило: Если у вас изначально имеется значение для переменной, то используйте инициализацию, вместо присваивания.

Неинициализированные переменные

В отличие от других языков программирования, языки Си и C++ не инициализируют переменные определенными значениями (например, нулем) по умолчанию. Поэтому, при создании переменной, ей присваивается ячейка в памяти, в которой уже может находиться какой-нибудь мусор! Переменная без значения (со стороны программиста или пользователя) называется **неинициализированной переменной**.

Использование неинициализированных переменных может привести к ошибкам, например:

```
#include <iostream>

int main()
{
    // Объявляем целочисленную переменную a

    int a;

    // Выводим значение переменной a на экран (a - это неинициализированная переменная)
    std::cout << a;

    return 0;
}
```

В этом случае компилятор присваивает переменной `a` ячейку в памяти, которая в данный момент свободна (не используется). Затем значение переменной `a` отправляется на вывод. Но что мы увидим на экране? Ничего, так как компилятор это не пропустит — выведется ошибка, что переменная `a` является неинициализированной. В более старых версиях Visual Studio компилятор вообще мог бы вывести какое-то некорректное значение (например, `7177728`, т.е. мусор), которое было бы содержимым той ячейки памяти, которую он присвоил нашей переменной.

Использование неинициализированных переменных является одной из самых распространенных ошибок начинающих программистов, но, к счастью, большинство современных компиляторов выдадут ошибку во время компиляции, если обнаружат неинициализированную переменную.

Хорошей практикой считается всегда инициализировать свои переменные. Это будет гарантией того, что ваша переменная всегда имеет определенное значение и вы не получите ошибку от компилятора.

Правило: Убедитесь, что все ваши переменные в программе имеют значения (либо через инициализацию, либо через операцию присваивания).

Тест

Какой результат выполнения следующих стейтментов?

```
int a = 6;
a = a - 3;
std::cout << a << std::endl; // #1

int b = a;
std::cout << b << std::endl; // #2

// В этом случае a + b является r-value
std::cout << a + b << std::endl; // #3

std::cout << a << std::endl; // #4

int c;
std::cout << c << std::endl; // #5
```

Ответы

Чтобы просмотреть ответ, кликните на него мышкой.

Ответ №1

Программа выведет `3`: `a - 3 = 3`, что и присваивается переменной `a`.

Ответ №2

Программа выведет `3`: переменной `b` присваивается значение переменной `a` (`3`).

Ответ №3

Программа выведет `6`: `a + b = 6`. Здесь не используется операция присваивания.

Ответ №4

Программа выведет `3`: значением переменной `a` до сих пор является `3`.

Ответ №5

Результатом будет ошибка, так как `c` — это неинициализированная переменная.